SASD GmbH

Admin-Handbuch

downloads-sasd / downloads.sasd.de

Installation, Konfiguration, Katalogpflege und Betriebsgrundlagen

Dokumenttyp	Administrationshandbuch / Betriebsdokument
Projekt	downloads-sasd
Ziel	Betrieb und Pflege der ersten MVC-nahen PHP-Version für downloads.sasd.de
Version	v0.1.0
Stand	2026-03-31
Repository	Robin-Goerlach/downloads-sasd

Einordnung

Dieses Handbuch beschreibt die erste produktionsnahe Projektbasis. Die Anwendung ist bewusst leichtgewichtig gehalten: kein Datenbankzwang, kein schweres Framework und dennoch eine saubere Trennung von Routing, Controller-, Service-, Repository-, View- und Katalogschicht.

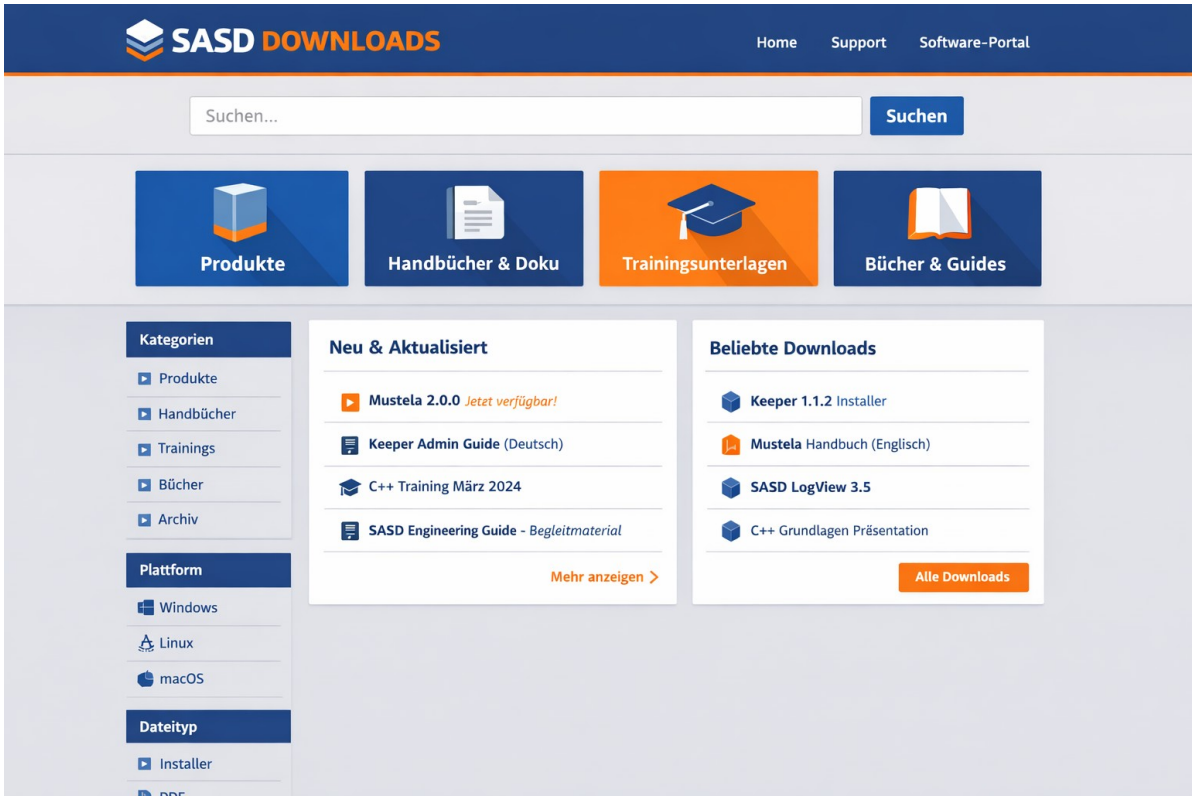

Abbildung 1 - Mockup der geplanten Startseite im SASD-Stil

Dokumentnutzung
Kapitel 4 beschreibt die empfohlene Installation unter Apache. Kapitel 7 regelt die laufende Katalogpflege, Kapitel 8 die Sprachvorbereitung und Kapitel 11 typische Fehlerbilder. Für Erstabnahmen genügen bereits die Schnellreferenz und die Abnahmepunkte im Anhang.

- Installation und Grundstart: Kapitel 4
- Konfiguration und Sprachsteuerung: Kapitel 6 und 8
- Produkte, Artefakte und Archivstatus pflegen: Kapitel 7
- Betrieb, Backup und Sicherheitsgrundlagen: Kapitel 9 und 10
- Schneller Funktionstest nach Änderungen: Anhang A und B

1. Zweck und Geltungsbereich

Dieses Handbuch dient als technische Grundlage für die Installation, Grundkonfiguration, Pflege und den laufenden Betrieb des Projekts downloads-sasd. Das Projekt bildet die erste Anwendungsbasis für die spätere Subdomäne downloads.sasd.de.

Die Anwendung ist als eigenständiges Download- und Artefaktportal konzipiert. Sie soll nicht nur Download-Links aus software.sasd.de bedienen, sondern auch selbständig nutzbar bleiben. Besucherinnen und Besucher müssen Produkte, Handbücher, Trainingsunterlagen, Begleitmaterialien und archivierte Versionen direkt über die Download-Domäne finden können.

Das Dokument richtet sich an Administratoren, Betreiber, technische Redakteure und Entwickler, die den Katalog pflegen, Artefakte veröffentlichen oder die Anwendung auf einen Server bringen.

Betriebsprinzip
Die aktuelle Version arbeitet dateibasiert. Fachinhalte werden in JSON-Katalogdateien gepflegt, Artefakte liegen im Dateisystem und ein generierter Gesamtkatalog sorgt für schnelle Laufzeiten. Die Architektur ist bewusst so vorbereitet, dass später primär die Repository-Schicht gegen eine Datenbankimplementierung ausgetauscht werden kann.

2. Systemüberblick

downloads-sasd ist eine kleine PHP-8.2-Anwendung mit Front Controller, eigener Request-/Response-Abstraktion, leichtgewichtigem Router, klaren Controllern und einer Service- und Repository-Schicht. Fachlich trennt das System Produktdaten von Artefaktdaten und löst Mehrsprachigkeit zentral.

Im Gegensatz zu software.sasd.de steht hier nicht das Marketing, sondern die Distribution im Vordergrund. Deshalb sind Suchzugang, Filterung, Produktseiten, Artefaktdetailseiten und direkte Download-Endpunkte von Anfang an vorgesehen.

Baustein	Rolle	Aktuelle Ausprägung
HTTP-Schicht	Routing und Auslieferung	Front Controller in public/index.php Eigener Router mit Platzhaltern
Controller	Anwendungsfälle pro Seitentyp	HomeController ProductController ArtifactController
Services	Fachlogik und Katalogsicht	CatalogService
Repository	Datenzugriff abstrahieren	JsonProductRepository JsonArtifactRepository
Katalog	Pflegbare Datenhaltung	data/catalog/source/*.json optional generierter catalog.json
Mehrsprachigkeit	UI-Texte und Locale-Auflösung	resources/lang/*.php LocaleResolver / Translator
Artefakt-Storage	Physische Dateien	storage/files/...

2.1 Wichtige HTTP-Endpunkte

Pfad	Zweck	Bemerkung
/	Startseite	Zentrale Suche, Einstiege und Highlights
/products	Produktübersicht	Listet Produkte unabhängig von software.sasd.de
/products/{slug}	Produktdetailseite	Zeigt verknüpfte Artefakte
/artifacts	Katalog- und Suchseite	Suche, Filter und Ergebnisliste
/artifacts/{slug}	Artefaktdetailseite	Dateiinfo, Metadaten und Download
/download/{slug}	Direkter Download	Liefert die physische Datei aus

3. Systemvoraussetzungen

Für die erste Version werden keine Datenbank und kein externer Suchdienst benötigt. Die kleinste stabile Zielumgebung ist daher bewusst unkompliziert gehalten.

Bereich	Mindestanforderung	Hinweis
PHP	PHP 8.2	Laut composer.json zwingend erforderlich
Webserver	Apache 2.4+ mit mod_rewrite	Empfohlen, da .htaccess bereits vorbereitet ist
Dateisystem	Leserechte auf storage/files	Für Downloads notwendig
CLI	php auf Shell-Ebene	Für validate-catalog.php und rebuild-catalog.php
Datenbank	nicht erforderlich	Die erste Version ist vollständig dateibasiert
Composer	optional	Für diese Projektbasis nicht zum Laufzeitstart nötig

Wichtiger Praxishinweis
Die Projektbasis ist im aktuellen Zustand primär für einen Webserver mit Rewrite-Unterstützung gedacht. Für einen realistischen Test der Pfade wird Apache mit DocumentRoot auf public empfohlen. Der reine PHP-Built-in-Server ist ohne zusätzliches Router-Skript nicht der bevorzugte Weg für diese Version.

4. Empfohlene Installation

Die folgende Installationsbeschreibung ist für Linux/Apache formuliert, weil dies am besten zu der vorhandenen .htaccess-Regelung und zur vorgesehenen öffentlichen Verzeichnisstruktur passt.

4.1 Projekt ablegen

Legen Sie das Projekt in ein eigenes Zielverzeichnis, zum Beispiel unter /var/www/downloads-sasd, und stellen Sie sicher, dass der Webserver ausschließlich auf das Unterverzeichnis public zeigt.

```
sudo mkdir -p /var/www/downloads-sasd
sudo rsync -a ./downloads-sasd/ /var/www/downloads-sasd/
```

4.2 Apache vorbereiten

Aktivieren Sie mod_rewrite und richten Sie einen Virtual Host ein, dessen DocumentRoot auf das Verzeichnis public zeigt. Dadurch bleiben Quellcode, Konfiguration und Rohdaten außerhalb des direkt ausgelieferten Webpfads.

```
sudo a2enmod rewrite
sudo systemctl reload apache2
```

```
<VirtualHost *:80>
  ServerName downloads.sasd.de
```

```
    DocumentRoot /var/www/downloads-sasd/public

    <Directory /var/www/downloads-sasd/public>
        AllowOverride All
        Require all granted
        Options FollowSymLinks
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/downloads-sasd-error.log
    CustomLog ${APACHE_LOG_DIR}/downloads-sasd-access.log combined
</VirtualHost>
```

4.3 Rechte und Besitz sauber setzen

Das Webfrontend benötigt Leserechte auf den Quellcode, die generierten Katalogdaten und die Artefaktdateien. Schreibrechte werden nur dort benötigt, wo Katalogdateien oder generierte Daten tatsächlich aktualisiert werden.

```
sudo chown -R www-data:www-data /var/www/downloads-sasd
sudo find /var/www/downloads-sasd -type d -exec chmod 755 {} \;
sudo find /var/www/downloads-sasd -type f -exec chmod 644 {} \;
```

Empfehlung für produktive Umgebungen
Für sauberen Betrieb ist ein Deployment-Benutzer besser als pauschale Webserver-Schreibrechte. Wenn Katalog-Rebuilds nicht aus dem Webprozess heraus erfolgen, sollten nur die wirklich benötigten Verzeichnisse schreibbar sein.

4.4 Katalog prüfen und generieren

Vor dem ersten Start sollte die Katalogquelle validiert und anschließend ein frischer Gesamtkatalog erzeugt werden.

```
cd /var/www/downloads-sasd
php cli/validate-catalog.php
php cli/rebuild-catalog.php
```

4.5 Inbetriebnahme verifizieren

Prüfen Sie danach die zentralen Pfade im Browser und laden Sie mindestens ein Testartefakt herunter. Für den Ersttest sollten Startseite, Produktübersicht, Artefaktkatalog und ein direkter Download erfolgreich sein.

- https://downloads.sasd.de/
- https://downloads.sasd.de/products
- https://downloads.sasd.de/artifacts
- https://downloads.sasd.de/artifacts/mustela-2-0-0-package
- https://downloads.sasd.de/download/mustela-2-0-0-package

5. Verzeichnisstruktur und Betriebsmodell

Pfad	Bedeutung	Administrative Relevanz

public/	öffentlicher Webroot	Enthält Front Controller, CSS und .htaccess
app/Bootstrap.php	Anwendungsverdrahtung	Lädt Konfiguration, baut Router und Services auf
src/	Anwendungscode	Controller, Service, Repository, Domain und Infrastruktur
config/app.php	Globale Konfiguration	Locales, Brand-Farben und relative Pfade
resources/lang/	UI-Sprachdateien	Deutsch und Englisch, später beliebig erweiterbar
resources/views/	PHP-Templates	Layout, Startseite, Produkt- und Artefaktseiten
data/catalog/source/	redaktionelle Rohdaten	products.json und artifacts.json werden manuell gepflegt
data/catalog/generated/catalog.json	generierter Gesamtkatalog	Wird per CLI neu erzeugt und beschleunigt den Betrieb
storage/files/	physische Download-Dateien	ZIP, PDF, Handbücher, Trainingsunterlagen und Archivdateien
cli/	Betriebsskripte	Validierung und Rebuild

Das entscheidende Architekturprinzip lautet: Inhalte werden zuerst fachlich im Katalog beschrieben und die eigentlichen Dateien separat abgelegt. Dadurch bleiben Navigation, Suche und Download-Auslieferung konsistent.

6. Konfiguration

Die zentrale Konfigurationsdatei ist config/app.php. Dort werden Name, Standard- und Fallback-Sprache, aktivierte und geplante Locales, Brand-Farben sowie die relativen Pfade der Anwendung definiert.

default_locale	Standardsprache der Oberfläche; aktuell de
fallback_locale	Fällt auf en zurück, wenn Inhalte in der aktiven Sprache fehlen
enabled_locales	Sprachen im Umschalter; aktuell de und en
planned_locales	Vorbereitete spätere Sprachen, zum Beispiel fr, es, pt, it, pl, tr, ar, hi, ko, zh und ja
brand	Farben für Templates und späteres Theming
paths	Physische Zielpfade für Views, Langdateien, Katalog und Storage

6.1 Sprachvorbereitung

Die Mehrsprachigkeit ist bewusst zweistufig aufgebaut. UI-Texte der Anwendung liegen in separaten Sprachdateien, während fachliche Inhalte direkt in den JSON-Katalogdaten mehrsprachig hinterlegt werden.

Ebene	Ort	Zweck
Oberfläche	resources/lang/de.php resources/lang/en.php	Navigation, Labels, Statusbegriffe und feste UI-Texte
Fachinhalte	data/catalog/source/*.json	Titel, Zusammenfassungen und Beschreibungen von Produkten und Artefakten
Auflösung	LocaleResolver / Translator	Ermittelt Sprache zentral und stellt Fallback sicher

7. Katalogpflege und Inhalte

Die erste Version wird bewusst nicht über eine Administrationsoberfläche gepflegt. Neue Inhalte werden kontrolliert im Dateisystem und in den JSON-Quellen erfasst. Genau dieser einfache Workflow macht die V1 schnell, nachvollziehbar und Git-tauglich.

7.1 Neues Produkt anlegen

- Produkt in data/catalog/source/products.json anlegen: id, slug, title, summary, description, productPageUrl, featured und tags.
- Für mehrsprachige Felder mindestens de und en befüllen.
- Falls bereits Artefakte vorliegen, Produktordner unter storage/files/products/<slug>/<version>/ vorbereiten.
- Danach passende Artefakte in artifacts.json ergänzen, validieren und den generierten Katalog neu bauen.

7.2 Neues Artefakt veröffentlichen

- Datei physisch in storage/files/... ablegen. Die bestehende Struktur nach Produkttyp, Produkt und Version sollte beibehalten werden.
- Metadaten in data/catalog/source/artifacts.json eintragen.
- Dabei insbesondere slug, group, type, version, language, platform, status, releaseDate und path korrekt setzen.
- Anschließend php cli/validate-catalog.php und php cli/rebuild-catalog.php ausführen.
- Zum Schluss Artefaktdetailseite und Download-Endpunkt im Browser testen.

```
{
  "id": "mustela-2.0.0-package",
  "slug": "mustela-2-0-0-package",
  "productSlug": "mustela",
  "group": "products",
  "type": "package",
  "title": { "de": "Mustela 2.0.0 Paket", "en": "Mustela 2.0.0 package" },
  "description": { "de": "Aktuelles Paket ...", "en": "Current package ..." },
  "version": "2.0.0",
  "language": "multi",
  "platform": "neutral",
  "status": "current",
  "releaseDate": "2026-03-28",
```

```
  "path": "products/mustela/2.0.0/mustela-2.0.0.zip"
}
```

7.3 Inhalte ausblenden, archivieren oder entfernen

Für die erste Version sollten Inhalte nicht vorschnell physisch gelöscht werden. Das Statusmodell ist dafür da, fachliche Sichtbarkeit und tatsächliche Datenlöschung zu trennen.

Status	Empfohlene Verwendung	Wirkung
current	Aktuelle produktive Version	Normal sichtbar und suchbar
lts	Langfristig unterstützte Version	Sichtbar; kann später gesondert hervorgehoben werden
deprecated	Noch vorhanden, aber abzulösen	Sichtbar mit fachlichem Warncharakter
archived	Historisch, aber noch relevant	Für Archivzwecke im Katalog verbleibend
hidden	Temporär unsichtbar	Für spätere Nutzung vorbereitet, aber nicht im normalen Katalog

Physisches Löschen sollte erst erfolgen, wenn klar ist, dass keine öffentliche URL, kein Projektverweis und kein Archivzweck mehr existieren.

8. Mehrsprachigkeit im Betrieb

Deutsch und Englisch sind in der ersten Version sofort einsatzbereit. Die Architektur ist jedoch bereits auf weitere Sprachen vorbereitet. Für spätere Internationalisierung müssen nicht Routing oder Controller neu gedacht werden, sondern im Wesentlichen nur Sprachdateien und Katalogeinträge ergänzt werden.

Wichtig ist die saubere Trennung: feste UI-Elemente in resources/lang, fachliche Inhalte im JSON-Katalog. Dieses Muster sollte beibehalten werden, weil es die spätere Datenbankmigration und Übersetzungsarbeit deutlich erleichtert.

8.1 Zusätzliche Sprache aktivieren

- Neue Sprachdatei in resources/lang anlegen, zum Beispiel fr.php oder ja.php.
- Die neue Locale in config/app.php unter enabled_locales ergänzen.
- Produkte und Artefakte in den JSON-Dateien um die neuen Sprachschlüssel erweitern.
- Danach die Oberfläche sowie repräsentative Produkt- und Artefaktseiten testen.

> **Sprachcodes sauber wählen**
> Für Hindi sollte intern hi verwendet werden. Für Chinesisch ist langfristig meist eine genauere Trennung in zh-CN und zh-TW sinnvoll. Arabisch und ostasiatische Schriften sollten später zusätzlich mit Blick auf Schriftarten, Umbruch und Leserichtung getestet werden.

9. Betrieb, Wartung und Backup

Aufgabe	Empfehlung	Rhythmus

Katalog validieren	Vor jeder Veröffentlichung ausführen	bei jeder fachlichen Änderung
Generierten Katalog neu bauen	Nach Katalog- oder Dateianpassungen	bei jeder Veröffentlichung
Browser-Funktionstest	Startseite, Suche, Produkt, Artefakt, Download prüfen	bei jeder Veröffentlichung
Dateisicherung	Repository, data/catalog und storage/files sichern	täglich oder releasebasiert
Integritätsprüfung	Prüfsummen der Artefakte überwachen	regelmäßig
Log- und Error-Review	Webserver- und PHP-Fehler prüfen	regelmäßig

Für Backups reicht in der ersten Version ein konsequentes Sicherungskonzept für Repository, Katalogdaten und Artefaktdateien. Eine Datenbank ist nicht vorhanden; dadurch ist die Wiederherstellung einfacher, darf aber organisatorisch nicht vernachlässigt werden.

9.1 Empfohlener Veröffentlichungsablauf

- Neue oder geänderte Dateien in storage/files ablegen.
- Katalogdaten in products.json und artifacts.json anpassen.
- Lokale Prüfung via validate-catalog.php.
- Generierten Katalog neu erzeugen.
- Fachlicher Browser-Test der wichtigsten Pfade.
- Git Commit, Tag bzw. Release-Dokumentation und Deployment.

10. Sicherheits- und Produktionshinweise

Der öffentliche DocumentRoot muss strikt auf public begrenzt bleiben. Konfigurationsdateien, Quellcode, Katalogquellen und Speicherdateien dürfen nicht direkt als Webroot exponiert werden.

Die erste Version lädt Dateien direkt aus dem Dateisystem aus. Deshalb müssen Pfade und Dateinamen ausschließlich aus vertrauenswürdigen Katalogdaten stammen. Der vorhandene Download-Controller arbeitet bereits über Slugs und nicht über freie Benutzerpfade, was die Angriffsfläche reduziert.

Der Status hidden sollte für sensible oder noch nicht freigegebene Artefakte verwendet werden, solange diese nicht öffentlich sichtbar sein sollen.

> **Empfohlene nächste Härtungsschritte**
> TLS, Security Header, saubere Fehlerseiten ohne technische Details, getrennte Deployments für Test und Produktion, Prüfsummenbereitstellung, optionale Signaturen sowie später Audit Logging und Download-Statistiken.

11. Troubleshooting

Symptom	Wahrscheinliche Ursache	Empfohlene Maßnahme
404 auf allen Pfaden	DocumentRoot zeigt nicht auf public oder Rewrite fehlt	Apache-Konfiguration und mod_rewrite prüfen
Artefaktseite sichtbar, Download schlägt fehl	Datei im Storage fehlt oder Pfad stimmt nicht	artifacts.json und storage/files vergleichen
validate-catalog.php meldet Fehler	Pflichtfelder oder Datei fehlen	JSON-Eintrag und reale Datei korrigieren
Änderungen erscheinen nicht	generierter Katalog nicht neu gebaut	rebuild-catalog.php ausführen
Sprache fällt auf Englisch zurück	aktive Locale in Inhalt nicht vorhanden	fehlende Übersetzung in JSON oder Sprachdatei ergänzen
Leere Oberfläche oder 500-Fehler	PHP-Fehler in Code oder Konfiguration	Webserver-/PHP-Logs prüfen und fehlerhafte Änderung rückgängig machen

12. Perspektive und Migrationspfad

Die aktuelle Projektbasis ist keine Sackgasse. Sie ist bewusst so strukturiert, dass der spätere Umstieg auf eine Datenbank vor allem die Repository-Schicht und eventuell Such- bzw. Paginierungslogik betrifft. Routing, Controller, Views und große Teile der Fachlogik können erhalten bleiben.

Ebenso kann später eine kleine interne Redaktionsoberfläche ergänzt werden, ohne die bestehende öffentliche Anwendung grundsätzlich neu zu schreiben. Der dateibasierte Start ist deshalb kein Provisorium, sondern eine absichtlich einfache erste Betriebsstufe.

Anhang A - Schnellreferenz

Validierung	php cli/validate-catalog.php
Katalog neu erzeugen	php cli/rebuild-catalog.php
Produkte pflegen	data/catalog/source/products.json
Artefakte pflegen	data/catalog/source/artifacts.json
Generierter Katalog	data/catalog/generated/catalog.json
Artefaktdateien	storage/files/...
UI-Sprachen	resources/lang/*.php
Globale Konfiguration	config/app.php
Öffentlicher Einstieg	public/index.php

Anhang B - Empfohlene erste Abnahmepunkte

- Startseite lädt vollständig und im richtigen SASD-Farbschema.
- Produkte und Artefakte werden mit deutschen und englischen Inhalten korrekt angezeigt.
- Suche und Filter auf /artifacts liefern sinnvolle Ergebnisse.

- Direkter Download über /download/{slug} funktioniert für repräsentative Dateien.
- Katalogvalidierung läuft fehlerfrei.
- Ein zusätzlicher Testeintrag kann veröffentlicht und bei Bedarf wieder archiviert werden.

Dokumentpflege

Dieses Handbuch sollte gemeinsam mit Änderungen an Routing, Konfiguration, Katalogschema, Betriebsabläufen und Sprachunterstützung fortgeschrieben werden. Empfehlenswert ist eine Überprüfung spätestens bei jedem neuen Release-Tag, bei Infrastrukturänderungen und vor der Einführung einer späteren Admin-Oberfläche oder Datenbankmigration.